Predicting Risk, Pricing the Future

Marshall School of Business, University of Southern California

DSO 530: Applied Modern Statistical Learning Methods

Professor Paromita Dubey

May 9, 2026

Group: Section 20261 / 16221 - 3

Jace Kang

Sameen Yasar

Binbo Xu

Wilhelm Osselmann-Chai

Jose Garcia

Executive Summary

This project addresses a pricing and underwriting question: how can an insurer estimate claim likelihood and expected loss before renewal so that premiums are fair, competitive, and profitable? We modeled three targets: CS for claim occurrence, LC for loss cost, and HALC for historically adjusted loss cost. Together, these targets allow the insurer to estimate both the likelihood of a claim and the expected financial risk of each policyholder.

The main business problem is mispricing. If high-risk policyholders are underpriced, the insurer may not collect enough premium to cover future losses. If low-risk policyholders are overpriced, those safer customers may leave for competitors. Over time, this can create adverse selection, where the insurer loses profitable low-risk customers and retains more expensive high-risk customers.

Target	Final model	Validation result	Selection rationale
LC	Tweedie XGBoost	Validation MSE = 252,117	Lower MSE than Random Forest; appropriate for zero-heavy, skewed positive losses.
HALC	Tweedie XGBoost	Validation MSE = 997,698	Best validation MSE for adjusted high-loss severity; captures nonlinear severity drivers.
CS	Random Forest Classifier	Validation ROC-AUC = 0.8323	Slightly higher ROC-AUC than XGBoost; stable risk-ranking performance under imbalance.

Our final modeling strategy was target-specific. For LC and HALC, we selected Tweedie XGBoost because both loss outcomes are zero-heavy and highly right-skewed. This structure is common in insurance data because many policyholders have zero losses, while a smaller group has large positive losses. For CS, we selected a Random Forest classifier because it produced the strongest validation ROC-AUC for claim-risk ranking under class imbalance.

- Business value: predicted CS supports claim-risk ranking, while LC and HALC support risk-adjusted pricing and break-even premium logic.
- Interpretation: top drivers include policy duration, payment method, nonpayment cancellation history, renewal timing, and net premium.
- Implementation output: the final submission file contains three columns exactly as required: LC, HALC, and CS.

Bottom line is the models convert policy, customer, payment, and vehicle signals into actionable risk estimates that help the insurer move from one-size-fits-all pricing to more precise, proactive, and defensible premium setting.

1. Business Question and Problem Framework

Insurance companies must set premiums before knowing which policyholders will file claims or how costly those claims will be. This creates a pricing problem. If premiums are too low for high-risk policyholders, the insurer may lose money. If premiums are too high for low-risk policyholders, safer customers may leave for competitors. Therefore, the goal of this project is to predict policyholder risk more accurately before claims occur.

We frame the project as a three-part prediction problem:

Target	Meaning	Construction	Business use
CS	Claim status	1 if X.16 > 0, else 0	Whether a policy is likely to generate a claim.
LC	Loss cost	X.15 / X.16 when claims exist, else 0	Expected loss per claim/policy for pricing decisions.
HALC	High adjusted loss cost	LC × X.18 when claims exist, else 0	Adjusted severity signal for break-even and high-loss pricing.

The model outputs serve different decisions. CS is a claim-likelihood signal that can support underwriting review and risk ranking. LC estimates expected loss and is directly relevant to premium setting. HALC adjusts LC using historical claim behavior, giving the insurer an additional risk-adjusted loss estimate. HALC should not be treated as the final premium by itself because the final premium would also include expenses, profit margin, taxes, and other business costs

2. Data Preparation and Exploratory Findings

The training set contains 39,928 policies and the test set contains 13,310 policies. We first constructed the target variables from observed claim columns in the training data. After creating LC, HALC, and CS, we dropped claim outcome fields from the model features to avoid data leakage. This was important because variables such as total claim cost and number of claims are used to define the targets, but they would not be available for new policyholders at prediction time.

We also transformed the date fields into usable year, month, and day components. In addition, we engineered policy_duration, which measures the time between policy start and next renewal. This feature was created because raw dates are difficult for the model to interpret directly, while policy duration has a clear business meaning related to coverage length, customer stability, and exposure.

- Class imbalance is material: 88.9% of training policies have no claim, so accuracy alone is not a reliable CS metric.

- A constant model that predicts no claim for every policyholder would appear accurate, but it would provide no risk ranking and no pricing differentiation.
- Risk type, payment behavior, renewal timing, policy duration, and premium-related variables showed meaningful predictive signals.

3. Modeling Approaches Considered

We considered both simple baselines and flexible tree-based ensemble models. Simple baseline approaches included constant mean prediction for LC and HALC and majority-class prediction for CS. These baselines are useful for context because they show what happens when every policyholder is treated similarly. However, they do not provide policyholder-level risk differentiation, which is the main business need in insurance pricing.

We therefore focused on tree-based ensemble models that can capture nonlinear relationships among policy, payment, customer, and vehicle features. The main models considered were Random Forest and XGBoost.

Random Forest was used as a strong benchmark because it handles nonlinear relationships and interactions well. It combines many decision trees, which generally makes it more stable than a single tree. Random Forest is also useful for both regression and classification tasks.

XGBoost was considered because it is often strong for structured tabular prediction problems. Unlike Random Forest, which builds many independent trees, XGBoost builds trees sequentially so that each new tree corrects errors made by previous trees. This can improve predictive performance when tuned properly.

For LC and HALC, we also considered the distribution of insurance losses. Both targets are zero-heavy and right-skewed, meaning many policyholders have zero losses while a smaller number have large positive losses. Because of this, we used Tweedie XGBoost for the final LC and HALC models. The Tweedie objective is appropriate for insurance-style loss outcomes because it can handle a combination of many zeros and positive continuous values.

For CS, we compared Random Forest and XGBoost classifiers using predicted probabilities rather than only hard classifications. This was important because the business goal is not simply to label policyholders as claim or no claim. The insurer also needs a risk score that ranks policyholders from lower to higher claim likelihood.

4. Final Model Selection

Target	Final model	Validation result	Selection rationale
LC	Tweedie XGBoost	Validation MSE = 252,117	Lower MSE than Random Forest; appropriate for zero-heavy, skewed positive losses.
HALC	Tweedie XGBoost	Validation MSE = 997,698	Best validation MSE for adjusted high-loss severity;

			captures nonlinear severity drivers.
CS	Random Forest Classifier	Validation ROC-AUC = 0.8323	Slightly higher ROC-AUC than XGBoost; stable risk-ranking performance under imbalance.

The selected models reflect the different statistical structure of each target. For LC, Tweedie XGBoost had the strongest validation performance. Its validation MSE was approximately 252,117, which corresponds to an RMSE of about 502. RMSE is easier to interpret because it is in the original scale of the target. This means the LC prediction errors were approximately 502 units in RMSE terms on the validation set.

For HALC, Tweedie XGBoost also performed best. Its validation MSE was approximately 997,698, corresponding to an RMSE of about 999. This RMSE is higher than LC because HALC is more variable and can take larger values. Hence, the higher RMSE does not automatically mean the model is worse; it reflects that HALC is a more difficult and larger-scale target.

For CS, Random Forest achieved the strongest validation ROC-AUC at 0.8323. ROC-AUC was used instead of accuracy because the claim class is rare. A model that predicts no claim for everyone could appear accurate, but it would fail to identify high-risk policyholders. ROC-AUC is more appropriate because it measures how well the model ranks actual claim customers above no-claim customers across different thresholds.

Although XGBoost was competitive for CS, Random Forest produced the highest validation ROC-AUC and offered stable probability-based ranking under class imbalance. Thus, we selected Random Forest for the classification target and prioritized validation performance over using one consistent algorithm across all three outcomes.

5. Model Interpretation and Innovation

Model interpretation was based on tree gain feature importance and supporting EDA. For LC and HALC, important drivers included policy duration, payment method, nonpayment cancellations, renewal timing, and net premium. These drivers are business-plausible because they capture customer stability, payment behavior, policy tenure, and exposure level.

The main innovation beyond the required analysis was the decision-oriented modeling framework. Instead of applying one model uniformly across all targets, we used target-specific models based on the statistical structure of each outcome. LC and HALC were modeled with Tweedie-optimized XGBoost to account for zero-heavy and right-skewed insurance loss data, while CS was modeled separately as a claim-risk ranking problem. We also connected the three outputs to business decisions: CS supports underwriting review, LC supports pricing adjustment, and HALC provides an adjusted severity lens for retention and break-even pricing logic. Finally, the feature-importance layer made the model more explainable by showing which customer, policy, payment, and renewal signals were driving predicted risk.

6. Limitations and Conclusions

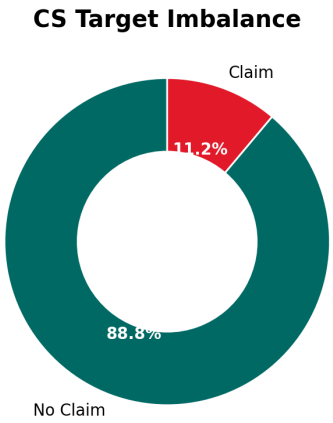
The models should be used as decision-support tools, not as fully automated underwriting decisions. Feature importance is predictive rather than causal, and the final CS threshold should depend on business costs such as review capacity, missed-claim losses, and customer fairness concerns. Because of time constraints, interpretation relied on XGBoost to gain importance and support EDA rather than full SHAP decomposition. Future work should use SHAP values to explain both global feature importance and individual policy-level predictions, along with calibration checks and cost-sensitive threshold optimization.

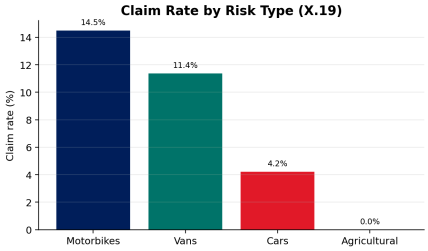
Overall, the final approach provides a coherent pricing framework: CS identifies claim likelihood, LC estimates expected loss, and HALC adds an adjusted severity lens. Together, these outputs help the insurer price more accurately, protect low-risk customers, and focus review effort on policies with higher expected risk. The practical value of the framework is that it gives the insurer a repeatable way to rank policyholders by claim likelihood, estimate expected loss, and make pricing decisions that are more precise than one-size-fits-all premium setting.

Technical Appendix

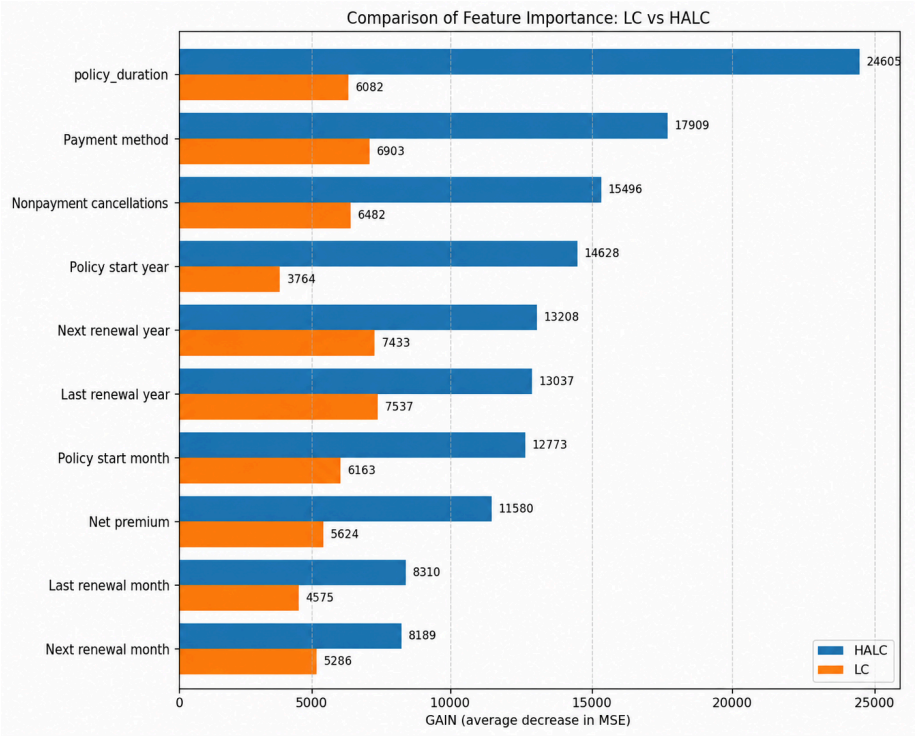
Appendix material is included to support the conclusions in the six-page main report.

Appendix A. EDA Figures





Appendix B. LC vs HALC Feature Importance



Feature importance uses XGBoost gain, which measures the average improvement in split quality contributed by a feature. It is useful for interpretation but should not be read as causal evidence.

Appendix C. Key Code Snippets

```
# Target construction
train_df["LC"] = np.where(train_df["X.16"] > 0, train_df["X.15"] / train_df["X.16"], 0)
train_df["HALC"] = np.where(train_df["X.16"] > 0, train_df["LC"] * train_df["X.18"], 0)
train_df["CS"] = (train_df["X.16"] > 0).astype(int)

# Leakage prevention for LC/HALC models
drop_cols_train = date_cols + ["X.15", "X.16", "X.17", "X.18", "LC", "HALC", "CS"]
X = lc_train_df.drop(columns=drop_cols_train, errors="ignore")

# Final model families
LC/HALC: XGBRegressor(objective="reg:tweedie", enable_categorical=True)
CS: RandomForestClassifier with probability output evaluated by ROC-AUC
```

Appendix D. Submission Output

The required prediction CSV was generated with exactly three columns: LC, HALC, and CS. The file follows the required group naming convention used throughout the project outputs.

Section_20261_16221_3_prediction

LC	HALC	CS
17.224213	44.742786	0.054026209431982400
9.104258	11.00335	0.04522564038199080
21.529358	20.468943	0.13195040029028100
37.459274	51.260155	0.13651335644629800
0.82188934	6.922039	0.01444196443475090
64.73061	97.00142	0.1191882028006000
32.961246	48.30317	0.18590739152990900
44.312366	133.6664	0.18448219719964000
6.08302	23.427402	0.04205560537379360
95.11694	117.56889	0.14211154383208200
4.284364	10.151531	0.07343826976875770
67.065155	85.68505	0.21651666633681500
6.3335223	8.030993	0.05185366879554180
3.7933326	9.594868	0.02243804117346100
68.85181	154.82367	0.09646488708410110
13.413751	29.584425	0.06439015479377310
4.1727324	6.1154284	0.03985743051539590
51.183487	81.638916	0.13435605001372200
22.069021	24.783985	0.11343706277130600
0.8662744	3.024823	0.026361501230257000
8.097749	7.147894	0.034737981329637900
70.6809	139.98228	0.26926689578072700
9.139278	54.198586	0.037843767719497300
54.139885	97.18958	0.13108511924513300
19.630615	18.82747	0.0900141613846552

*First 25 rows of predicted LC, HALC, and CS